

TP14

Bypass de la détection root avec Frida, Objection et Medusa

Rapport de TP

Réalisé par EZBIRI Amira

Avertissement éthique. Ce TP doit être exécuté uniquement sur des appareils ou applications appartenant au testeur, ou dans le cadre d'une autorisation écrite (audit, recherche, enseignement). Les techniques de root bypass et d'instrumentation dynamique servent à comprendre et analyser une application, pas à contourner des protections en production.

Table des matières

1	Objectif du TP	2
2	Environnement et outils	2
2.1	Outils utilisés	2
2.2	Vérification initiale de l'environnement	2
3	Analyse du comportement de l'application	3
4	Bypass avec Frida	4
4.1	Principe général	5
4.2	Exécution Frida	8
5	Bypass avec Objection	9
5.1	Pourquoi Objection	9
5.2	Observation dans la console	9
6	Bypass avec Medusa	10
7	Résultat obtenu	11
8	Dépannage	11
9	Conclusion	12

1 Objectif du TP

L'application *OWASP Uncrackable1* intègre plusieurs mécanismes de défense contre l'analyse :

- détection de root ;
- détection d'émulateur ;
- détection de hooks ou d'instrumentation dynamique ;
- arrêt de l'application dès qu'une condition suspecte est rencontrée.

L'objectif du TP est de neutraliser ces contrôles avec plusieurs approches complémentaires :

- **Frida** pour installer des hooks en mémoire ;
- **Objection** comme surcouche pratique de Frida ;
- **Medusa** pour automatiser certains bypass ;
- **ADB** pour lancer, vérifier et observer le comportement de l'application.

2 Environnement et outils

Le TP repose sur un poste PC Windows avec ADB et Python, ainsi que sur un appareil Android ou un émulateur connecté en USB.

2.1 Outils utilisés

Outil	Rôle dans le TP
ADB	Connexion au terminal Android, lancement de commandes et vérification de l'état du périphérique.
Frida	Instrumentation dynamique, injection de script et modification du comportement à l'exécution.
Objection	Interface plus simple pour appliquer des bypass prêts à l'emploi.
Medusa	Automatisation de certaines opérations Frida, selon la version disponible.
OWASP Uncrackable1	Application cible utilisée pour tester la détection de root.

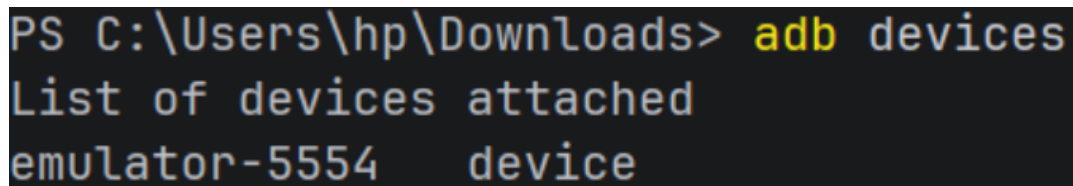
2.2 Vérification initiale de l'environnement

Avant toute manipulation, il faut s'assurer que Python, pip et ADB sont fonctionnels. La capture suivante montre le démarrage du serveur Frida sur l'appareil via ADB.

```
PS C:\Users\hp\Downloads> adb shell chmod 755 /data/local/tmp/frida-server/frida-server-17.9.1-android-x86
PS C:\Users\hp\Downloads> adb shell "/data/local/tmp/frida-server/frida-server-17.9.1-android-x86 &"
```

FIGURE 1 – Démarrage de frida-server sur l'appareil Android via ADB. Le binaire est rendu exécutable puis lancé en arrière-plan.

La commande `adb devices` confirme ensuite que le périphérique est bien détecté par le poste de travail.

A terminal window with a black background and white text. The prompt is 'PS C:\Users\hp\Downloads>'. The command 'adb devices' is entered. The output shows 'List of devices attached' followed by 'emulator-5554 device' on the next line.

```
PS C:\Users\hp\Downloads> adb devices
List of devices attached
emulator-5554    device
```

FIGURE 2 – Vérification ADB : l'appareil apparaît avec l'état device.

3 Analyse du comportement de l'application

Au lancement, l'application cible affiche son écran principal avec un champ de saisie et un bouton de vérification. Avant le bypass, une boîte de dialogue *Root detected!* apparaît immédiatement.

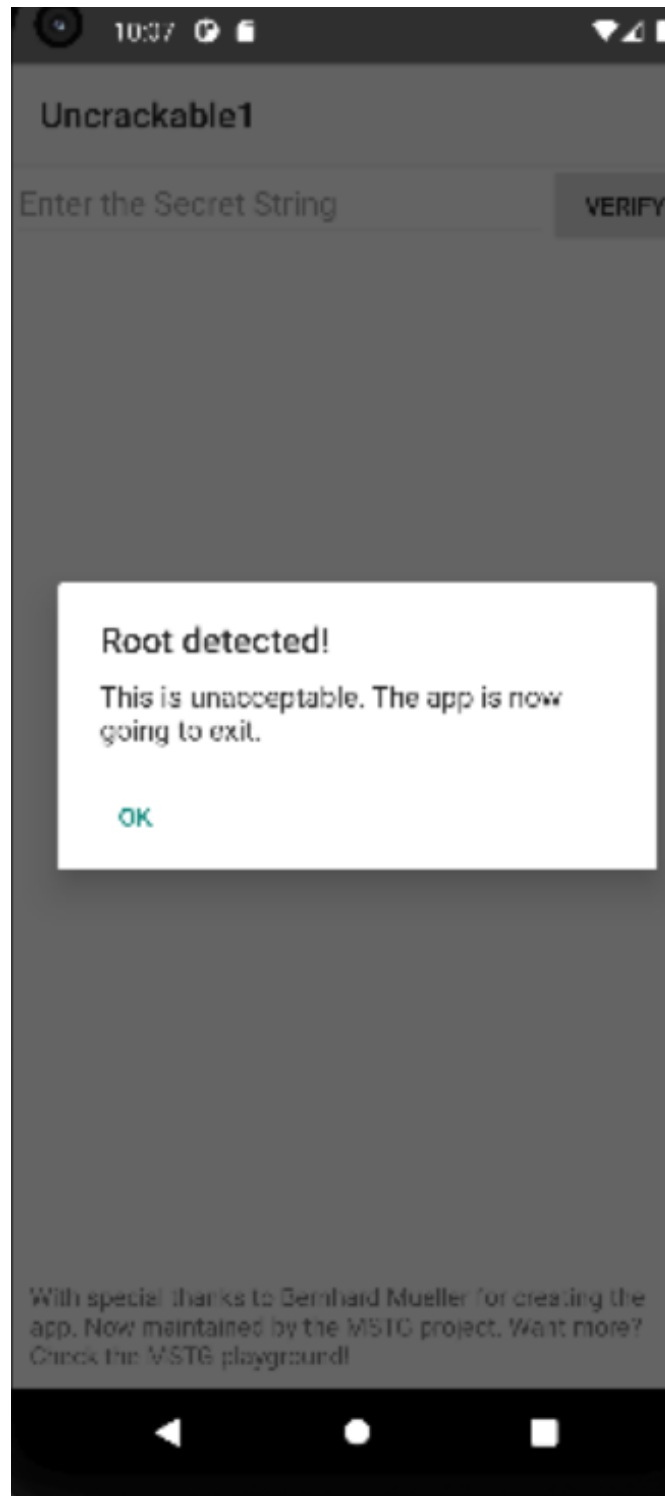


FIGURE 3 – L'application bloque son exécution et signale une détection de root.

Ce comportement confirme que l'app peut être utilisée comme support de TP, car elle contient un mécanisme de protection actif que l'on peut observer puis neutraliser.

4 Bypass avec Frida

Frida permet d'injecter un script au démarrage de l'application (*spawn*) ou sur un processus déjà lancé (*attach*). Dans ce TP, l'injection au démarrage est particulièrement utile car les checks de sécurité sont

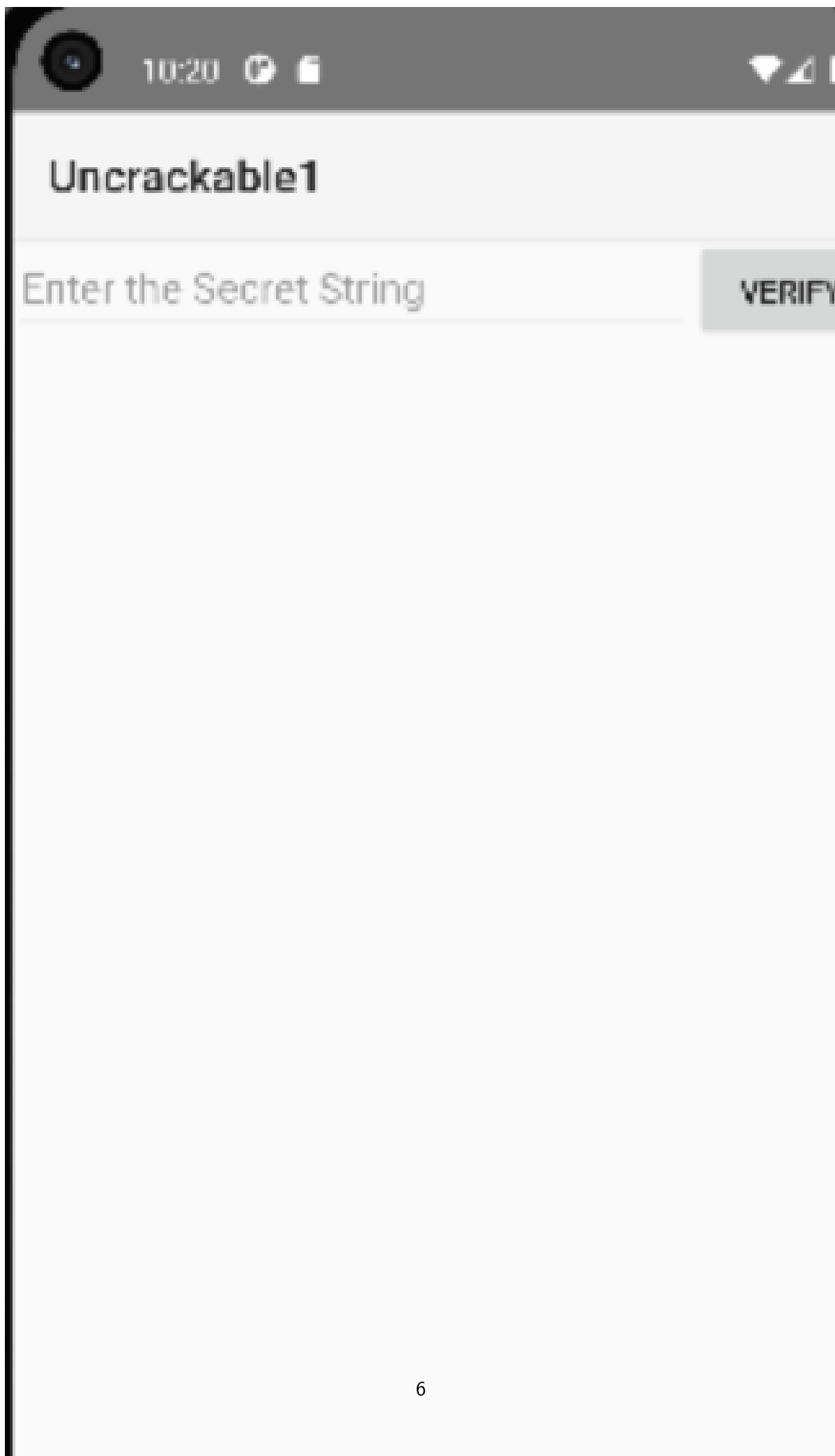
souvent exécutés très tôt.

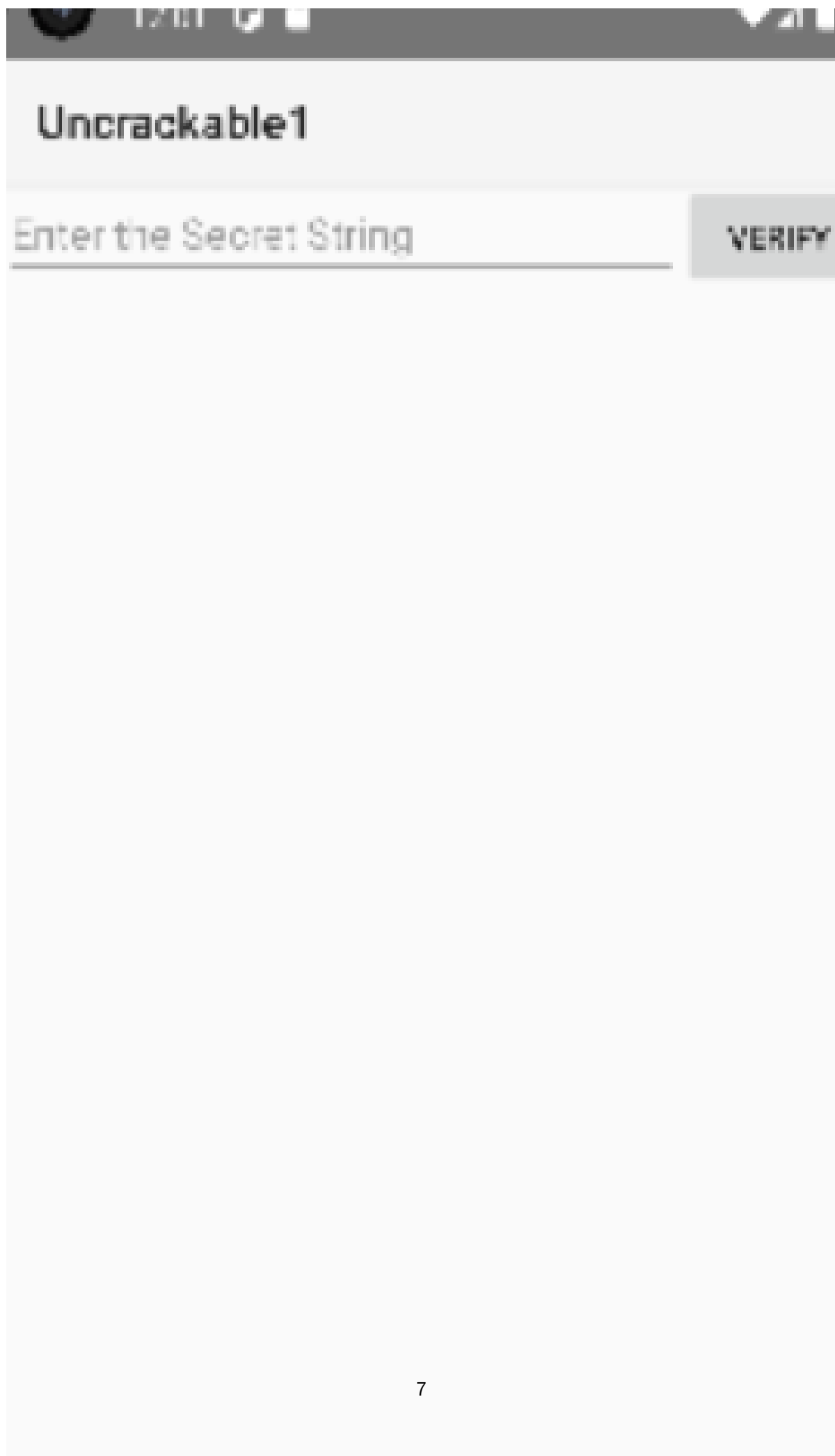
4.1 Principe général

Le principe consiste à intercepter les fonctions qui révèlent un environnement compromis :

- lecture de `Build.TAGS` ;
- vérification de fichiers suspects comme `/system/bin/su` ;
- appels à `Runtime.exec()` pour lancer `su` ou `which su` ;
- appels natifs éventuels vers `open`, `access`, `stat`.

Dans la console Frida, on observe que les hooks sont bien installés et que plusieurs chemins suspects sont bloqués.





4.2 Exécution Frida

La capture suivante montre Frida en mode *spawn* sur l'application cible, avec la confirmation des hooks installés.

```

  ____|____|____|____|____|____|____|____|____|____|
 |  .  |  .  |  -  |  -  |  -  |  -  |  .  |  |
 |_____|_____|_____|_____|_____|_____|_____|_____|
      |____|(object)inject(ion) v1.12.4

Runtime Mobile Exploration
  by: @leonjza from @sensepost

[tab] for command suggestions
pwasp.mstg.uncrackable1 (run) on (Android: 11) [usb] # android root disable
(agent) Registering job 975458. Name: root-detection-disable
pwasp.mstg.uncrackable1 (run) on (Android: 11) [usb] # jobs list
Job ID  Type  Name
-----
975458  hook  root-detection-disable
pwasp.mstg.uncrackable1 (run) on (Android: 11) [usb] # _

```

FIGURE 6 – Exemple de console d'instrumentation : la détection de root est neutralisée.

La sortie `frida-ps` -Uai permet ensuite de confirmer que l'appareil est correctement accessible et que Frida voit les processus installés.

```
PS C:\Users\hp> frida -U -f owasp.mstg.uncrackable1 -l "C:\\S2\\programming mobile\\bypass"
    /----|
    | C_ | |
    > _   |
    /_/ |_|
    . . . .
    . . . .
    . . . .
    . . . .
    . . . .
    . . . .
    . . . .
    More info at https://frida.re/docs/home/
    . . . .
    . . . .
    Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `owasp.mstg.uncrackable1`. Resuming main thread!
[Android Emulator 5554::owasp.mstg.uncrackable1 ]-> [+] Build.TAGS hooked
[+] File.exists hooked
[+] Runtime.exec hooked
[+] All hooks installed
[+] File.exists blocked: /system/bin/su
[+] File.exists blocked: /system/xbin/su
[+] File.exists blocked: /system/app/Superuser.apk
```

FIGURE 7 – Injection Frida : hooks installés sur l'application cible et blocage de chemins sensibles.

```
PS C:\WINDOWS\system32> frida-ps -Uai
PID  Name                               Identifier
-----
3901  Google                               com.google.android.googlequicksearchbox
7880  Phone                               com.google.android.dialer
7842  Photos                              com.google.android.apps.photos
1715  SIM Toolkit                          com.android.stk
8176  Settings                            com.android.settings
-    Calendar                           com.google.android.calendar
-    Camera                             com.android.camera2
-    Chrome                             com.android.chrome
-    Clock                              com.google.android.deskclock
-    Contacts                           com.google.android.contacts
-    Drive                              com.google.android.apps.docs
-    Files                              com.google.android.documentsui
-    Glasses                            com.google.android.glasses.companion
-    Gmail                              com.google.android.gm
-    Maps                               com.google.android.apps.maps
-    Messages                           com.google.android.apps.messaging
-    Personal Safety                    com.google.android.apps.safetyhub
-    YouTube                            com.google.android.youtube
-    YouTube Music                      com.google.android.apps.youtube.music
PS C:\WINDOWS\system32>
```

FIGURE 8 – Liste des applications et processus visibles avec `frida-ps -Uai`.

5 Bypass avec Objection

Objection simplifie l'usage de Frida grâce à des commandes prêtes à l'emploi. Pour ce TP, la commande clé est `android root disable`.

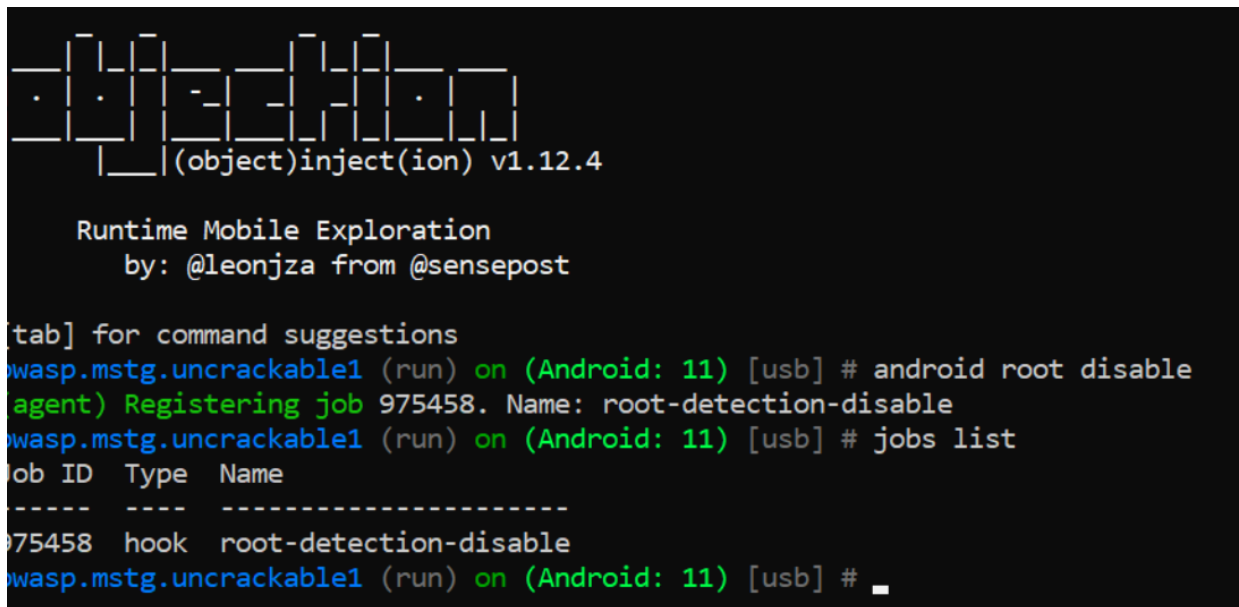
5.1 Pourquoi Objection

Objection permet de gagner du temps sur les scénarios classiques :

- modification de valeurs de sécurité courantes ;
- masquage de la présence de root ;
- bypass de vérifications Java fréquentes ;
- interface interactive adaptée aux tests rapides.

5.2 Observation dans la console

Après lancement de la commande, Objection enregistre un job de type *hook* qui correspond à la désactivation de la détection root.



```
(object)inject(ion) v1.12.4

Runtime Mobile Exploration
by: @leonjza from @sensepost

[tab] for command suggestions
wasp.mstg.uncrackable1 (run) on (Android: 11) [usb] # android root disable
(agent) Registering job 975458. Name: root-detection-disable
wasp.mstg.uncrackable1 (run) on (Android: 11) [usb] # jobs list
Job ID  Type  Name
-----
975458  hook  root-detection-disable
wasp.mstg.uncrackable1 (run) on (Android: 11) [usb] #
```

FIGURE 9 – Objection exécute android root disable et enregistre le hook associé.

6 Bypass avec Medusa

Medusa propose une autre manière d'appliquer des modules Frida. Dans les captures, l'outil analyse les propriétés de l'appareil puis liste les applications tierces installées.

```

N 1) Device(id="socket", name="Local Socket", type='remote')
2) Device(id="barebone", name="GDB Remote Stub", type='remote')
3) Device(id="emulator-5554", name="Android Emulator 5554", type='usb')

Enter the index of the device to use: 3

Device properties:

[ro.product.manufacturer]: [Google]\r
[ro.product.name]: [sdk_gphone_x86]\r
[ro.build.version.all_codenames]: [REL]\r
[ro.build.version.base_os]: []\r
[ro.build.version.codename]: [REL]\r
[ro.build.version.incremental]: [12134477]\r
> [ro.build.version.min_supported_target_sdk]: [23]\r
[ro.build.version.preview_sdk]: [0]\r
> [ro.build.version.preview_sdk_fingerprint]: [REL]\r
[ro.build.version.release]: [11]\r
> [ro.build.version.release_or_codename]: [11]\r
[ro.build.version.sdk]: [30]\r
> [ro.build.version.security_patch]: [2021-08-05]\r
[ro.build.id]: [RSR1.240422.006]\r
> [ro.build.tags]: [dev-keys]\r

- 3rd party installed applications:

> [0] owasp.mstg.uncrackable1
(emulator-5554) medusa>

```

FIGURE 10 – Medusa affiche les propriétés du périphérique et prépare l'instrumentation sur l'émulateur.

Cette étape est utile pour vérifier le profil de l'appareil :

- fabricant ;
- modèle ;
- version Android ;
- présence de dev-keys, souvent un indice d'émulateur ou de build de test.

7 Résultat obtenu

Après les opérations de bypass, l'application peut rester ouverte sans déclencher immédiatement le blocage lié au root. Cela permet d'étudier plus loin son comportement, de tester ses flux internes et d'observer le trafic ou les réponses affichées.

Le succès du TP repose donc sur trois points :

1. le périphérique Android est bien détecté ;
2. Frida et/ou Objection sont opérationnels ;
3. les protections de base contre le root sont neutralisées.

8 Dépannage

Quelques problèmes courants apparaissent souvent pendant ce type de laboratoire :

- **Frida ne se connecte pas** : vérifier la version du client et du serveur, puis relancer `frida-server` ;
- **ADB ne voit pas l'appareil** : vérifier le débogage USB, le câble et les autorisations ;
- **L'application crashe au spawn** : essayer un attach au lieu du spawn ;
- **Objection ne neutralise pas tout** : compléter avec un script Frida plus fin ou tester un autre point d'injection ;
- **Medusa ne détecte pas l'app cible** : vérifier l'architecture de l'appareil et les modules disponibles.

9 Conclusion

Ce TP montre comment une application Android peut essayer de résister à l'analyse, et comment plusieurs outils d'instrumentation permettent de reprendre le contrôle à des fins de test autorisé. Frida donne la flexibilité, Objection apporte la rapidité, et Medusa facilite certains scénarios récurrents. Les captures fournies documentent le passage d'un état bloqué par *root detection* à un état où l'application reste exploitable pour l'analyse.